Computer Science, University of Warwick
CS141
Functional Programming
λ GPT Report

Ian GITAU
IG: 5621825

7th April 2026

Module Coordinator: Alexander Dixon
Date Due: 7st April, 2026

1 Introduction

The aim of this coursework was to implement a small chatbot, λ GPT, in Haskell. The chatbot operates as a REPL, repeatedly reading input, parsing it into a structured request, and producing an appropriate response. The main focus of the coursework was to combine parser combinators, IO, and stateful behaviour in order to build an interactive functional program.

The final system supports several categories of request. It can respond to greetings, answer date-related questions, evaluate arithmetic expressions written in longhand English, remember facts between interactions, and use the word "that" to refer to the most recent arithmetic answer.

2 Design and Implementation

The methodology used for this coursework was to break the implementation into three main stages: parsing, handling, and repetition through the REPL loop. This was done deliberately in order to reduce complexity and to follow the programming practices discussed later in the module. Rather than placing all logic inside one large interactive function, the program was divided into smaller pieces where each part performs a more specific job.

The first stage is request parsing. Raw user input is first converted into a value of a custom Request data type. This allows the rest of the program to operate on structured values such as Hello, Today, HowLong year month day, Remember-Name name thing, and MathExpr raw. This design improves readability because later parts of the program do not have to repeatedly inspect raw strings. Instead, the parsing logic is centralised into a set of parser definitions.

The second stage is request handling. Once a line of input has been parsed into a Request, a corresponding response is generated. This handling stage also has access to the chatbot's memory, which consists of remembered facts and the most recent arithmetic result. This means that stateful features such as remembering information and interpreting the word "that" can be implemented without affecting the rest of the parser design.

The third stage is the REPL loop itself. A recursive loop was used rather than a simple forever loop, because memory needs to be passed from one interaction to the next. This allows the chatbot to remain interactive while preserving state between requests. The loop reads the input, parses it, handles it, prints the output, and then repeats with the updated memory.

This structure was chosen because it clearly separates the major concerns of the program. Parsing is responsible only for recognising sentence shapes, the response logic is responsible only for behaviour, and the loop is responsible only for control flow. This organisation made the code easier to debug and made the

design easier to justify.

3 Justification of Design Choices

Parser combinators were used because they fit the problem naturally and make the request formats clear in the code. This approach is more elegant than using a large number of manual string checks and is easier to justify in a coursework focused on parsing.

The arithmetic request stores the raw expression text during request parsing, rather than evaluating it immediately. This was necessary because the "that" feature depends on the previous arithmetic result. By delaying arithmetic evaluation until the response phase, the program can first substitute "that" and then evaluate the resulting expression correctly.

The memory was threaded manually through the loop rather than using a monad transformer. This was chosen because it kept the control flow easy to understand and made the implementation simpler to debug. Since only part of the chatbot needs memory, this approach was sufficient and easier to explain.

The code was also split into smaller helper functions for parsing and responding. This reduced repetition, improved readability, and better followed the principle of separation of concerns discussed in the lecture material.

4 Reflection

The most challenging part of the coursework was deciding where different responsibilities should sit, especially for arithmetic handling and memory. In particular, once the "that" feature was introduced, it became clear that expression parsing and request parsing should not be mixed too early. Refactoring the code so that arithmetic evaluation happened later improved both correctness and readability.

Another challenge was adding memory without making the code unnecessarily complicated. A recursive loop carrying explicit memory proved to be a practical solution and allowed the remembered facts and previous arithmetic answer to persist between requests.

Overall, the final program meets the core requirements of the coursework and demonstrates the use of parsing, IO, recursion, and basic stateful programming in a functional style.

5 Conclusion

This coursework produced a small but functional chatbot in Haskell. The final implementation supports greetings, date queries, arithmetic expressions, remem-

bered facts, and arithmetic recall using “that”. The project also showed the value of separating parsing, response generation, and state handling into distinct parts of the program.

If extended further, the program could support more flexible input phrasing, more advanced arithmetic, and richer memory. However, in its current form it already provides a clear and justifiable implementation of the required λ GPT functionality.

6 Reference

CS141 lecture materials and lab sheets
Hackage documentation for megaparsec
Hackage documentation for time
StackOverflow for techniques